

Agentic AI: Intelligent Task Execution System

MAJOR PROJECT REPORT

**Submitted in partial fulfillment of the requirement for the Degree of
Bachelors of Engineering in Computer Science & Engineering**

Submitted To:



[PARUL UNIVERSITY, VADODARA, GUJARAT (INDIA)]

Submitted By:

Shubham Paithane [2303051051245]

Adarsh Singh Rajput [2303051051092]

Under The Guidance of:

Mr. Meet Kumar Patel

ASSISTANT PROFESSOR

CSE

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PARUL INSTITUTE OF TECHNOLOGY VADODARA, GUJARAT
SESSION: AY 2026-2027**

Parul University
Parul Institute of Technology



(Session: 2026 -2027)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that **SHUBHAM PAITHANE, ADARSH RAJPUT** students of CSE VII Semester of “**Parul Institute of Technology, Vadodara**” have completed their Major Project titled “**Agentic AI – Autonomous Task Execution Assistant (Aizen)**”, as per the syllabus and has submitted a satisfactory report on this project as a partial fulfillment towards the award of degree of Bachelor of Technology in Computer Science and Engineering under Parul University, Vadodara, Gujarat (India).

Mr. Meet Kumar Patel
ASSISTANT PROFESSOR
CSE
Project Guide

Ms. Arnika Patel
CSE DEPARTMENT
PIT, Vadodara
Project Coordinator

Dr. Sumitra Menaria
Head (CSE)
PIT, Vadodara
Head of Department

External Examiner

Dr. Ruchi Srivastava
PRINCIPAL
PIT, Vadodara
Principal

DECLARATION

We the undersigned solemnly declare that the project report “**Agentic AI – Autonomous Task Execution Assistant (Aizen)**” is based on our own work carried out during the course of our study under the supervision of **Mr. Meet Kumar Patel, Assistant Prof., CSE, PIT.**

We assert the statements made and conclusions drawn are the outcomes of our own work. We further certify that

- The work contained in the report is original and has been done by us under the general supervision of our supervisor.
- The work has not been submitted to any other Institution for any other degree / diploma / certificate in this university or any other University of India or abroad.
- We have followed the guidelines provided by the university in writing the report.
- This report is an extension of our minor project “Agentic AI – Autonomous Task Execution Assistant”, upgraded and substantially rebuilt for the major project stage.

Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them in the text of the report and giving their details in the references.

SHUBHAM PAITHANE [2303051051245]

ADARSH SINGH RAJPUT [2303051051092]

ACKNOWLEDGEMENT

Behind any major work undertaken by an individual there lies the contribution of the people who helped him to cross all the hurdles to achieve his goal.

It gives us immense pleasure to express our sense of sincere gratitude towards our respected guide **Mr. Meet Kumar Patel**, Assistant Professor, for his persistent, outstanding, invaluable co-operation and guidance. It is our achievement to be guided under him. He is a constant source of encouragement and momentum that any intricacy becomes simple. We gained a lot of invaluable guidance and prompt suggestions from him during the entire project work. We will be indebted to him forever and we take pride to work under him.

We also express our deep sense of regards and thanks to **Dr. Sumitra Menaria**, Head of the Computer Science & Engineering Department, and to **Ms. Arnika Patel**, Project Coordinator, CSE Department, for their precious advice, guidance and leadership. We are also grateful to **Dr. Ruchi Srivastava**, Principal, Parul Institute of Technology, for providing the environment and resources that made this project possible.

Last but not the least, our humble thanks to the Almighty God.

Place: Vadodara

Date:

ABSTRACT

Most of the early time on any new coding project doesn't go toward the actual idea. It goes toward setup, picking a stack, creating the folder structure, writing config files, getting some basic version running before the real feature even starts. Autocomplete tools help inside a single file, sure, but someone still has to decide the project structure and put every piece together by hand. Our minor project proposed an **agentic assistant** for exactly this, something that could take one plain-English instruction and work out the steps on its own, instead of the user typing each step out separately. That version stayed mostly conceptual, though. For this major project we picked one specific area to actually build it in, generating and updating software projects, mainly because it's easy to grade honestly: either the project the agent built runs, or it doesn't.

We're calling the system **Aizen**, a web-based coding assistant that runs entirely in the browser. When a user types an instruction, the backend first works out what kind of request it's looking at through a simple intent classifier, a plain question, a request to build something new, or a request to change a project already sitting in the workspace. Build and refine requests get routed to a project-builder function, which asks a large language model (Groq, in our case) to return the project as a structured response, files with their content plus a short explanation, across the ten stacks we support, everything from a plain HTML page up to a full React and Flask application. Sometimes that model call just fails. When it does, a deterministic fallback takes over and produces a basic working template instead, so nobody ever gets handed back an empty response. Once a first draft exists, an agent loop picks up from there: listing files, reading one, rewriting one, running a validation command, cycling through a limited number of steps until it judges the result good enough, rather than generating once and calling it done.

Wrapped around that core loop sits an authenticated workspace, JWT-based login, per-user chat history, an in-browser Monaco code editor with a live preview for whatever gets generated, a sandboxed terminal for running commands, one-click GitHub push, a small template gallery, a usage dashboard, and a separate SQL Assistant panel that turns an English question into a SQL query. Python and Flask run the backend, React and Vite run the frontend, and SQLite holds user accounts and chat history.

This report covers why we upgraded from the minor project in the first place, a survey of similar agentic and code-generation tools and where Aizen actually sits among them, the system's

methodology and module breakdown, its system requirements, the interface as it stands right now, and the problems we ran into (and fixed) while building it, plus what's still left on the list.

INDEX

CHAPTER	TOPIC	PAGE NO.
Chapter 1	INTRODUCTION	11
1.1	Overview	11
1.2	Problem Statement	12
1.3	Objective of Project	13
1.4	Applications or Scope	14
1.5	Organization of Report	14
1.6	From Minor Project to Major Project	15
Chapter 2	LITERATURE SURVEY	17
Chapter 3	METHODOLOGY	21
3.1	Background / Overview of Methodology	21
3.2	Project Platforms used in Project	21
3.3	Proposed Methodology	22
3.4	Project Modules	24
3.5	Diagrams (Architecture, DFD, Use Case)	25
Chapter 4	SYSTEM REQUIREMENTS	29
4.1	Software Requirements	29
4.2	Hardware Requirements	29
Chapter 5	EXPECTED OUTCOMES...(with GUI)	31
Chapter 6	CONCLUSION & FUTURE SCOPE	36
6.1	Conclusion	36
6.2	Future Work	37
Chapter 7	REFERENCES	39

LIST OF TABLES

Table No.	Table Description	Page No.
Table 1.1	Minor project vs. major project	15
Table 4.1	Software requirements	29
Table 4.2	Client-side hardware requirements	29
Table 4.3	Server-side hardware requirements	30

LIST OF FIGURES

Figure No.	Figure Description	Page No.
Figure 3.1	System architecture of Aizen	26
Figure 3.2	DFD Level 0 - context diagram	26
Figure 3.3	DFD Level 1 - detailed process breakdown	27
Figure 3.4	Use case diagram - Aizen	28
Figure 5.1	Aizen landing page	31
Figure 5.2	Sign-up / login interface	32
Figure 5.3	Chat-based build workspace	33
Figure 5.4	Code editor and live preview	33
Figure 5.5	Analytics dashboard	34
Figure 5.6	SQL Assistant panel	35

LIST OF ABBREVIATIONS

S.No.	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CLI	Command Line Interface
4	CORS	Cross-Origin Resource Sharing
5	CSS	Cascading Style Sheets
6	DFD	Data Flow Diagram
7	GUI	Graphical User Interface
8	HTML	HyperText Markup Language
9	HTTP/HTTPS	HyperText Transfer Protocol (Secure)
10	IDE	Integrated Development Environment
11	JSON	JavaScript Object Notation
12	JWT	JSON Web Token
13	LLM	Large Language Model
14	REST	Representational State Transfer
15	SSE	Server-Sent Events
16	SQL	Structured Query Language
17	UI	User Interface
18	URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

1.1 Overview

Aizen lets a user finish a software task by typing one instruction in plain English instead of doing every step by hand. That's the short version. Underneath it sit two ideas we first floated back in our minor project, though in a much more general and mostly conceptual form there:

- **Autonomous Task Planning** – the agent works out what the user is actually asking for, and for a build or refine request, figures out which files it will need before writing a single one.
- **Autonomous Task Execution** – the agent writes the files into a workspace, then loops back over its own output, checking it, rerunning parts, fixing what's broken, rather than generating once and walking away.

"Task" in the minor project stayed pretty vague, honestly. This time we picked one domain and stuck to it: generating and maintaining software projects. Part of the reason is that it gives us an honest way to grade the agent afterward. Either the thing it built runs, or it doesn't. Not much room to fudge that kind of result.

We're using the word "agentic" on purpose here, not just because it's the term everyone reaches for this year. A regular AI assistant is reactive: you ask, it answers, it stops there. Breaking a goal into steps, deciding what each step should produce, stitching the results back together, all of that is still on the user. Give a system a goal instead of a single step, and it starts deciding its own sequence of actions, checking what one action produced before picking the next. That's roughly what Aizen does. The intent classifier works out what kind of request came in, the project builder turns that into a first set of files, and the tool-use loop lets the agent look back at what it just made, catch problems in it, and fix them, without anyone re-typing instructions or walking it through each step by hand. None of this terminology matters much to whoever's actually using it, though. From their side, they type one sentence and get back a project they can open and edit. Not a pile of code snippets they still have to assemble themselves.

One more thing worth saying plainly: what Aizen isn't. It's not a full IDE replacement. It isn't trying to match Copilot or Cursor feature-for-feature either, fewer languages, no enterprise tier, none of

that. The scope stays narrow on purpose. Take one instruction, produce or edit a small-to-medium multi-file project across a fixed set of stacks, and keep the entire loop visible inside one browser tab, from the instruction going in to a runnable result coming out. Narrow is exactly what let two of us finish this, test it, and actually understand every part of it inside our project timeline, while still touching every stage of a real agentic pipeline: classification, planning, generation, self-correction, delivery.

1.2 Problem Statement

Try to test an idea quickly as a student or a developer, and the same wall shows up almost every time, no matter how small the idea is. Pick a stack. Scaffold the project. Wire up the config files. Only after all of that do you get to the part that actually matters.

- **For individual users** – most AI assistants still answer one question at a time. Turning an idea into a runnable multi-file project means copying code out of a chat window by hand, file by file.
- **For students and hobby developers** – the tools that do generate whole projects, IDE-integrated agents, hosted app builders, tend to lock you into one editor, one subscription, one hosting platform. They rarely show you the reasoning behind what they built either.
- **For our own minor project** – the design never left the diagram stage. It proposed an agent loop, sure, but never tested it against anything narrow enough to actually check. So we genuinely didn't know whether the idea would survive contact with real prompts, real failures, real output.

Look closely at these three and they're really one gap wearing different clothes. For an individual user it's friction: the intelligence to write any single file already exists somewhere in a chat window, but assembling many of those answers into one working project falls entirely on the human. Students and hobbyists hit a different wall, lock-in. Whatever tool removes the setup friction usually ties you to one paid product and hides how anything actually gets generated underneath. And for us, our own minor project's problem was ambition nobody had bothered to check yet. Drawing a diagram of an agent that plans and executes, that part's easy. Actually building it, and watching it fail in ways the diagram never once predicted, is a completely different animal.

What we needed, then, was a system that takes one instruction, plans and generates a working project across more than one stack, lets the user see and edit whatever gets produced, and can recover when a generation step fails, all without ever leaving an ordinary browser tab.

1.3 Objective of Project

The objectives carried over and refined from the minor project are:

- To build an agent that reliably classifies a user's natural-language instruction into a conversational reply, a new build request, or a refinement of an existing project.
- To implement autonomous project generation across a meaningful range of technology stacks (React, Flask, Node/Express, plain HTML, Python, Java, C++ and PHP), rather than one hard-coded template.
- To give the agent a bounded tool-use loop, so it can inspect its own output (list files, read a file, rewrite a file, run a command) and improve a result instead of only producing it once.
- To keep the system usable when the underlying language model call fails, by falling back to a deterministic template so the user always receives something they can act on.
- To provide an integrated browser-based environment, code editor, live preview, execution sandbox and one-click GitHub deployment, so nothing has to be copied out of the chat window by hand.
- To show, with a working implementation instead of just diagrams, that the agentic approach we explored in the minor project actually holds up when tested.

Each objective above maps onto a specific part of the system covered later in this report: classification is Stage 2 in Section 3.3, multi-stack generation with its fallback is Stage 3 plus the module list in Section 3.4, the bounded tool-use loop is Stage 4, and the browser environment is basically all of Chapter 5. Keeping that mapping straight while we built turned out to matter. Any time we thought about adding something new, we could ask whether it actually served one of these stated goals, instead of bolting it on just because it seemed interesting at the time.

1.4 Applications or Scope

Aizen's core loop, one instruction in, a working project out, isn't tied to any single stack, so the same mechanism ends up useful in a handful of different settings:

- **Rapid Prototyping** – turns a one-line idea (“build me a Flask API with auth”) into a working starter project in the time it takes to describe it.
- **Education** – lets a student compare how the same request is answered across different stacks, and look at the generated project structure instead of just one isolated code snippet.
- **Hackathons and personal projects** – removes the setup overhead that otherwise eats into limited build time.
- **Developer tooling** – the intent-classification and tool-use loop are general enough that they could be reused as the base for other, more specialised agents later.
- **Quick proof-of-concept work** – a team trying to decide if an idea is worth building out fully can get something clickable and runnable in a few minutes instead of planning a whole sprint for it.

None of these cases require the user to already know which stack is "right" for the job, or to have a development environment set up before finding out whether an idea is worth chasing. We limited the scope here on purpose too. Production deployment, team collaboration, long-term maintenance, none of it is covered. It sits in Chapter 6 as future work, not something we're pretending already exists.

1.5 Organization of Report

The rest of this report breaks down like this:

- **Introduction** – background, problem statement and objectives.
- **Literature Survey** – related agentic frameworks and code-generation tools, and the gaps between them and this project.
- **Methodology** – platforms, proposed methodology, modules and system diagrams.
- **System Requirements** – hardware and software needed to run Aizen.
- **Expected Outcomes** – the interface and behaviour of the implemented system.

- **Conclusion & Future Scope** – summary of what was achieved and what is planned next.
- **References** – papers, documentation and tools referred to during the project.

1.6 From Minor Project to Major Project

Same title, different project in a lot of ways. Our minor project proposed an agentic assistant that could plan and execute a task from a single instruction, but it stopped at the design stage. The agent loop stayed conceptual. The intent handling stayed conceptual. There was nothing we could point to and say "here, this is what it actually does." And turning that proposal into Aizen wasn't just a matter of typing up the original diagrams as working code either, most of what's in Chapter 3 now, the bounded tool-use loop, the deterministic fallback, the SQL Assistant, none of it was in the original proposal. It showed up later, out of dealing with a model that kept failing in ways no diagram had accounted for. Table 1.1 lays out exactly what changed on the way here.

Table 1.1: Minor project vs. major project

Aspect	Minor Project	Major Project (this report)
Scope	A general-purpose task-execution agent, discussed at the level of an idea.	Narrowed specifically to software project generation and refinement, where success is checkable: the project either runs or it does not.
Implementation	Diagram-level design only; no working application was built or demonstrated.	A complete working web application, Aizen, built and used with real prompts.
Technology stack	Proposed only; no backend, frontend or database was implemented.	Flask + Groq LLM backend, React + Vite frontend, JWT authentication, SQLite, Monaco editor.
Agent behaviour	Single-pass planning described conceptually, with no bound on how it would behave.	A bounded tool-use loop (list, read, write, run) that inspects and corrects its own output over several steps.
Failure handling	Not addressed.	An explicit deterministic fallback plus visible debug logging when the underlying model call fails, found necessary after a real failure during development.
Diagrams	Generic DFD and use-case diagrams not tied to a specific implementation.	All diagrams redrawn to reflect Aizen's actual modules and data flow.
Evaluation	No way to test whether the agent "worked."	Directly testable: a build request either produces a running project or a clear, logged error explaining why

		not.
Feature set	Limited to the single task-execution loop described in the proposal; no auxiliary domain-specific assistants.	Adds a dedicated SQL Assistant that converts an English question into a runnable SQL query, built as its own panel and backend route without touching the core architecture.

CHAPTER 2

LITERATURE SURVEY

Purpose: to look at existing agentic-AI research and code-generation tools that relate to building a system which plans and executes multi-file software projects from one instruction, and to work out what's still missing from them.

1. Background: why agentic AI matters

A normal AI assistant is reactive. Nothing more complicated than that: it answers what you asked, then stops. Agentic AI works differently. Instead of reacting to a single instruction, it holds onto a goal, decides for itself what sequence of actions gets it there, and needs far less babysitting along the way. We touched on this idea in fairly general terms during the minor project. Here, in the major project, we're finally applying it to something concrete: software generation.

And this isn't just a theoretical point. Give someone a reactive assistant and they end up doing all the planning themselves, breaking their own goal into sub-steps, asking for each one in turn, then mentally stitching the results back together afterward. That works fine for one well-defined question. It stops working the moment the request becomes "build me a working project," because that one sentence is hiding a dozen smaller decisions inside it, what stack to use, how the files should be laid out, keeping every file consistent with every other file, checking that the whole thing actually runs at the end. An agentic system takes that breakdown work off the user's plate. Which is why, going through this survey, the question we kept circling back to wasn't "is this tool smart", it was how much of that planning burden each one actually removes.

2. Major categories of agentic-AI approaches

Broadly, agents fall into a few categories, and each one trades flexibility against predictability differently:

- **Rule-based / script-driven agents** – older automation that relies on fixed decision trees and pre-written scripts to run through a sequence of steps. Predictable and cheap to run, but they can't handle an instruction that wasn't already anticipated when the rules were written.

- **LLM-powered agents** – a large language model reads the instruction, comes up with a plan, and calls tools as needed instead of following a fixed script. Aizen falls in this category: both the intent classifier and the project builder lean on a hosted LLM to make sense of free-form instructions, something a fixed rule set could never fully cover.
- **ReAct (Reason + Act)** – mixes short reasoning steps with tool calls, so the agent's next move is based on what the last one returned (Yao et al., 2022). Our tool-use loop (Section 3.3, Stage 4) works on the same principle: at every step, the model sees what its last action returned before choosing the next one.
- **Tool-augmented LLMs** – frameworks like LangChain and AutoGPT let a model call external APIs, browse the web, or write files as part of doing a task. They support a wide range of possible tools. We went the other way and hard-coded a small fixed set, list, read, write, run, specific to project generation; Section 6 talks about this as a gap rather than a strength.

3. Code-generation-specific tools

Since this project narrows the general agentic idea down to software engineering, it makes more sense to compare it against tools built for that narrower job, not just general-purpose agent frameworks:

- **GitHub Copilot** – strong at line-level and function-level autocomplete inside an editor, but it does not plan or create a multi-file project structure on its own; the developer still owns the scaffolding.
- **Cursor and similar IDE-integrated agents** – can apply multi-file edits, but require the specific editor to be installed and are usually tied to a paid subscription.
- **Hosted prompt-to-app builders (e.g. v0.dev, bolt.new)** – generate a working front-end quickly but are generally tied to one stack and to the vendor's own hosting, with limited visibility into the generation loop itself.
- **Devin and comparable “autonomous software engineer” demos** – show that a long-running autonomous coding agent is possible, but access is limited and the internal architecture isn't published in detail.

Put these together and a pattern shows up fast: the closer a tool gets to actual autonomous, multi-file software generation, the more likely it is closed-source, paywalled, or locked to one vendor's editor and hosting. Not much sits in the open, inspectable middle, where a student can actually read through the planning and self-correction steps as they happen. That's roughly where this project tries to sit.

4. Frameworks and general-purpose building blocks

We also looked at a few general-purpose agent frameworks while designing our own tool-use loop, even without using any of them directly:

- **LangChain** – a widely used framework for combining LLM calls with tool use and memory. Its “agent executor” pattern, looping a model over a fixed tool set until it decides it's done, is close to the shape of our own loop. We built ours directly rather than through the framework, mainly so we could actually trace what was happening at each step ourselves.
- **AutoGPT / BabyAGI** – early demos of a model looping over its own output to chase a goal with very little human input. Watching these made it clear that an unbounded loop is unreliable in practice, which is the direct reason we capped ours at a fixed number of iterations instead of trusting the model to decide when to stop.
- **OpenAI Assistants API** – gives built-in thread memory and tool-calling on top of a hosted model.
- **Semantic Kernel (Microsoft)** – an SDK for building planning and memory around an LLM inside enterprise applications.

5. Challenges identified in the literature

A handful of failure modes came up again and again across these tools, and any agentic system, ours included, has to be built around them rather than hoping they won't happen:

- **Hallucination** – a model can confidently return a plan, a file, or a tool call that is just plain wrong.

- **Infinite or repetitive loops** – an unbounded agent loop can get stuck repeating the same action forever without making any real progress. It's why we capped ours at a fixed number of iterations rather than trusting the model to know when to stop.
- **Context window limits** – a long enough conversation, or a large enough generated project, eventually outgrows what one model call can see in a single pass.
- **Silent failure** – plenty of these tools stay quiet when the model call underneath them has actually failed. This one bit us directly during development, it wasn't some hypothetical risk described in a paper, it happened to us (see Chapter 6).
- **Evaluation difficulty** – grading open-ended agentic behaviour is genuinely hard. "Did it build something reasonable" just doesn't score the way a plain right-or-wrong answer does.

Out of that whole list, silent failure is the one we can actually speak to firsthand rather than secondhand from someone else's paper. Chapter 6 walks through exactly how it happened to us.

6. Identified gaps and how this project addresses them

Almost everything we looked at falls into one of two buckets. Either it's general-purpose and not really built for the specific, checkable job of producing a runnable project, or it's a code-generation tool that locks you into one editor, one stack, one hosting setup. What seems to be missing is a browser-based assistant that stays stack-agnostic, carries a single instruction all the way from planning through to a runnable, editable project, and fails gracefully rather than silently when a model call breaks somewhere in the middle. Filling that gap is what Aizen is actually attempting: ten stacks behind one interface, the tool-use loop and validation step both kept visible in the chat log, and an explicit fallback wired in for whenever generation fails.

We'd rather not overstate any of this. Aizen isn't beating Copilot or Cursor on raw code quality, and it was never trying to. The claim we're actually making is smaller than that: stack-agnostic generation, a visible bounded self-correction loop, and a fallback that fails loudly instead of silently, all three living inside one lightweight browser-based application. "Better coding assistant" would be a much bigger claim. This one, at least, is something we can back up with what we actually built.

CHAPTER 3

METHODOLOGY

3.1 Background / Overview of Methodology

There's exactly one entry point into Aizen. A greeting, a build instruction, a follow-up change, it doesn't matter which, every request hits the same endpoint and runs through an intent classifier before anything else happens. From there the classifier sorts the request onto one of three paths: an ordinary conversational reply, a new project build, or a refinement of something already sitting in the user's workspace. That wasn't an accidental design choice. One entry point made the whole system easier to reason about while we were building it, and later on it's exactly what made the failure described in Chapter 6 traceable, since every request has to pass through the same handful of stages before it ever reaches the language model.

There was an easier option available: two separate, explicit buttons, one for "build a project," one for "ask a question," which would have made a classifier unnecessary in the first place. We didn't go that way because it just pushes the problem raised back in Chapter 2 onto the user instead, they'd need to know which category their request falls into before they've even started typing. A single text box that figures out the intent on its own feels more natural to use, but it also puts a lot more pressure on the classifier actually getting it right, which is exactly why Section 3.3 treats classification as its own dedicated step instead of quietly folding it into project generation.

3.2 Project Platforms used in Project

- **Frontend:** React 19 running on Vite 6, with Zustand handling state and Tailwind CSS v4 for styling. Monaco Editor powers the in-browser code editing, react-markdown with syntax highlighting renders the chat output, recharts drives the analytics view, and lucide-react supplies the icons.
- **Backend:** Python 3 on Flask, with Flask-CORS handling cross-origin requests. Session tokens run through PyJWT, passwords go through Werkzeug's bcrypt-based hashing, and SQLite stores accounts and chat history.
- **AI integration:** Groq's hosted chat-completion API, which handles both the conversational path and the structured project-generation prompts.

- **Tools:** Visual Studio Code for writing it, Git and GitHub for version control, and the browser's own developer tools for debugging the SSE stream.

None of these choices came down to what's popular right now. React with Vite made more sense than a heavier framework here, Aizen is one long-lived page rather than a multi-route site that needs server-side rendering, so Vite's fast dev server and simpler build setup fit better than the extra machinery something like Next.js would bring along. Flask won out over Django for a similar reason: the backend really amounts to a handful of REST and streaming endpoints wrapped around one agent loop, not a full content-management application, so Flask's smaller footprint kept the codebase something two people could realistically manage. SQLite got chosen over a client-server database because we're running one local Flask process per user rather than a hosted multi-tenant service, which is also exactly why Chapter 6 lists it as a scaling limitation. Groq got picked mainly for its response speed on chat-completion calls, and that matters more than it might sound like, Stage 4's tool-use loop fires off several model calls back-to-back inside a single response the user is sitting there waiting on.

3.3 Proposed Methodology

Six stages take the request from the moment the user hits enter to the moment a runnable project lands in their workspace:

1. User Instruction Intake

The user types a natural-language instruction into the chat box, and it goes to the backend along with a short slice of the recent conversation so the agent has some memory of what came before. There are no separate form fields, the instruction goes through exactly as typed, and that's what lets one text box handle chat, build and refine requests without making the user pick a mode first.

2. Intent Classification

A lightweight classifier built on keywords and patterns, not a model call, sorts the instruction into a greeting or general question (chat), a request to build something new (create), or a request to change the current workspace (refine). That choice to keep this step model-free was deliberate. It stays fast, and it doesn't depend on the AI provider being reachable at all, so even if Groq's API happens to be down for a moment, the system still knows what kind of request just came in. It simply can't act on it yet.

3. Response Generation (chat path) or Project Generation (build / refine path)

A chat-classified request has the backend send the instruction, a short system prompt and the recent conversation memory to the Groq API and stream the reply back to the browser. A build request works differently, a separate project-builder function asks the model for a structured response, project name, blueprint label, a short explanation, and a list of files with their content, and those files get written into a per-session workspace on disk. Sometimes that model call just fails, the provider's unreachable, the response comes back malformed, some exception nobody thought to plan for. When that happens, a deterministic template generator takes over and produces a small but complete starter project matched to whichever blueprint it detected, so the user never ends up staring at an empty response. Refine requests take the same path, except they also get handed the workspace's current contents as context first, letting the model edit what's already there rather than starting over from a blank slate.

4. Agent Tool-Use Loop

Once there's an initial scaffold, the agent enters a bounded loop, capped at six to eight iterations, where at each step it picks one of four actions: list the files in the workspace, read a specific file, rewrite a file, or run a short validation command. Every step, the model is asked whether it wants one more action or whether the result is final. That loop is the difference between Aizen checking its own work and Aizen just generating something once and walking away, it can catch an incomplete file and fix it before the reply ever goes out. The iteration cap exists because of the infinite-loop problem raised in Chapter 2, without one, a model that keeps deciding "one more action" has no reason to ever hand control back.

5. Validation

Before the result goes back, the backend runs one safe, allow-listed command drawn from the generated project's own suggested run commands, checking that a script actually parses, for instance, and records what happened as a validation note shown next to the generated files. It's a light check, not a full test suite. Its only job is catching the most obvious failure, a file that won't even parse, cheaply and fast. It doesn't guarantee the generated project is fully correct.

6. Result Delivery

The final explanation, the generated or updated files, and a manifest describing the project, its blueprint, its files, the validation outcome, stream back to the browser over Server-Sent Events and show up in the chat as an inline artifact, with the files also sitting ready in the code editor, live preview and terminal panels. Streaming beats waiting for everything to finish because a build request can take several seconds once the tool-use loop works through a few iterations, and watching it stream in is what tells the user the system is actually doing something rather than stuck.

3.4 Project Modules

- **Authentication Module** – sign-up and login, JWT issuance, bcrypt password hashing, per-user chat history.
- **Intent Classification Module** – routes each instruction to chat, create, or refine.
- **Project Builder Module** – model-backed generation across ten blueprints (React app, Flask API, static landing page, Python CLI, full-stack React+Flask, Node/Express API, Java CLI, C++ CLI, PHP site, and a minimal starter); each blueprint also has its own deterministic template fallback.
- **Agent Tool-Execution Module** – the bounded list/read/write/run loop, used to both polish a freshly generated project and apply targeted edits to one that already exists.
- **Workspace & File Management Module** – materialises generated files on disk per session and maintains the project manifest.
- **Live Preview & Code Editor Module** – a Monaco-based editor with save support and an iframe preview for HTML-renderable projects.
- **Execution / Terminal Module** – runs a chosen file's code in a sandboxed process and returns its output.
- **Search Module** – a client-side search across generated file names and contents.
- **GitHub Deployment Module** – pushes the current workspace to a GitHub repository.
- **Template Gallery Module** – a small set of curated starting points a user can launch directly.

- **Analytics Module** – tracks how many projects and files have been generated and in which languages, shown as simple charts.
- **SQL Assistant Module** – a dedicated panel that takes a plain-English question, along with an optional table schema and target dialect, and turns it into a runnable SQL query with a plain-English explanation. Rather than fine-tune a separate model on a question-to-SQL dataset, we reused the same Groq-hosted model and narrowed it down to this one job with a dedicated system prompt and a strict output format.

None of these twelve modules is a separately deployed service, Aizen is one Flask application and one React application, and the list above is about dividing responsibility inside that single codebase, not splitting it across processes or servers. That grouping still paid off during development, we could build and test one piece, authentication say, without needing the project-builder or agent-loop pieces working yet.

3.5 Diagrams

System Architecture

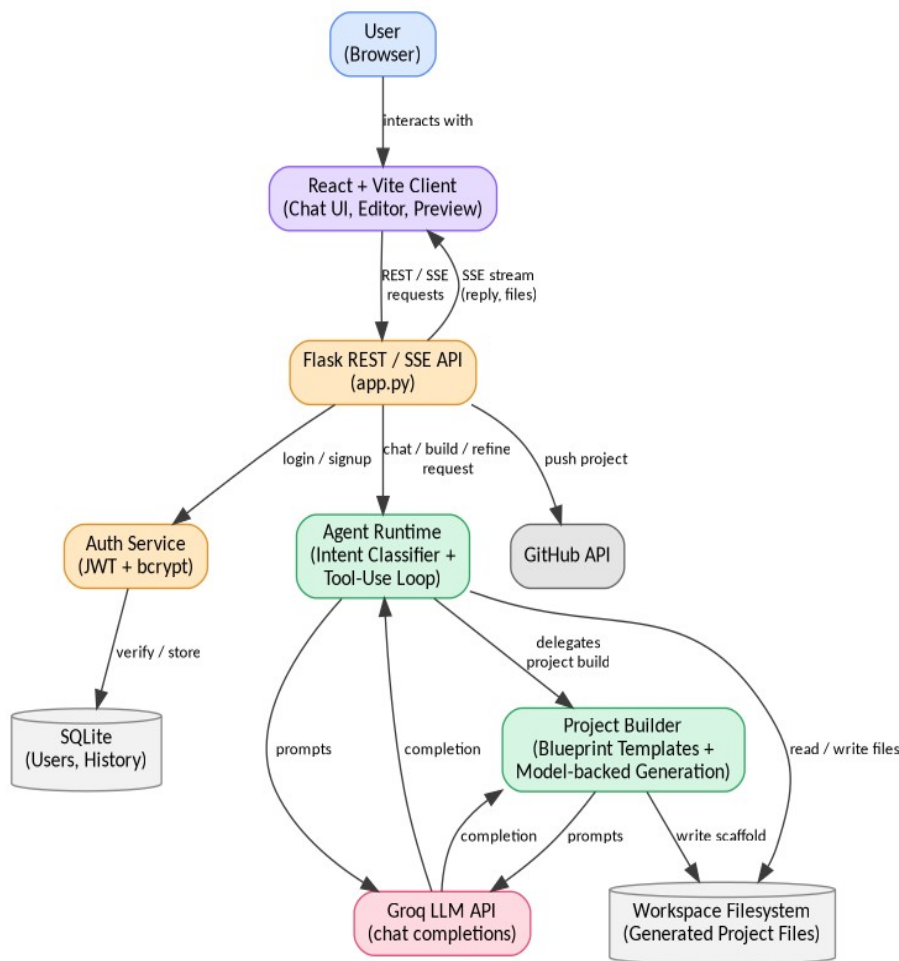


Figure 3.1: System architecture of Aizen

DFD Level 0 (Context Diagram)



Figure 3.2: DFD Level 0 – context diagram

DFD Level 1

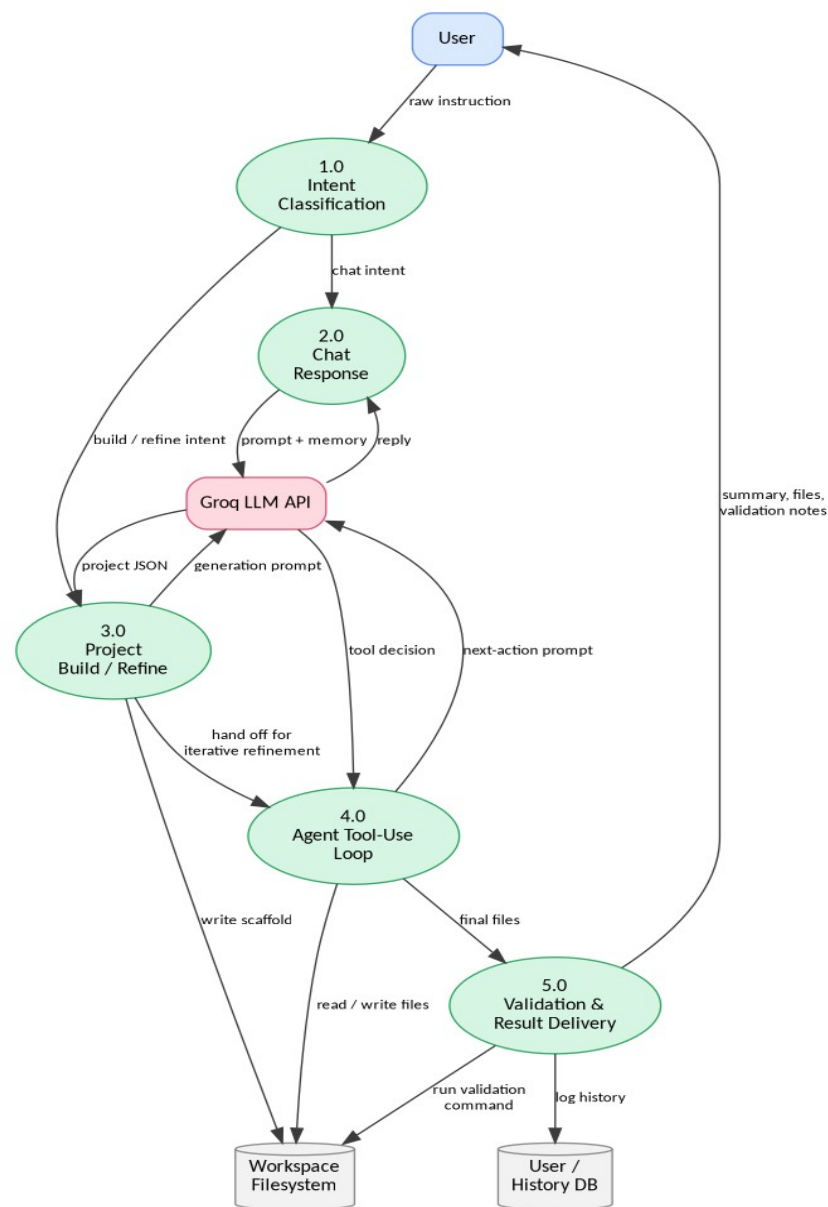


Figure 3.3: DFD Level 1 – detailed process breakdown

Use Case Diagram

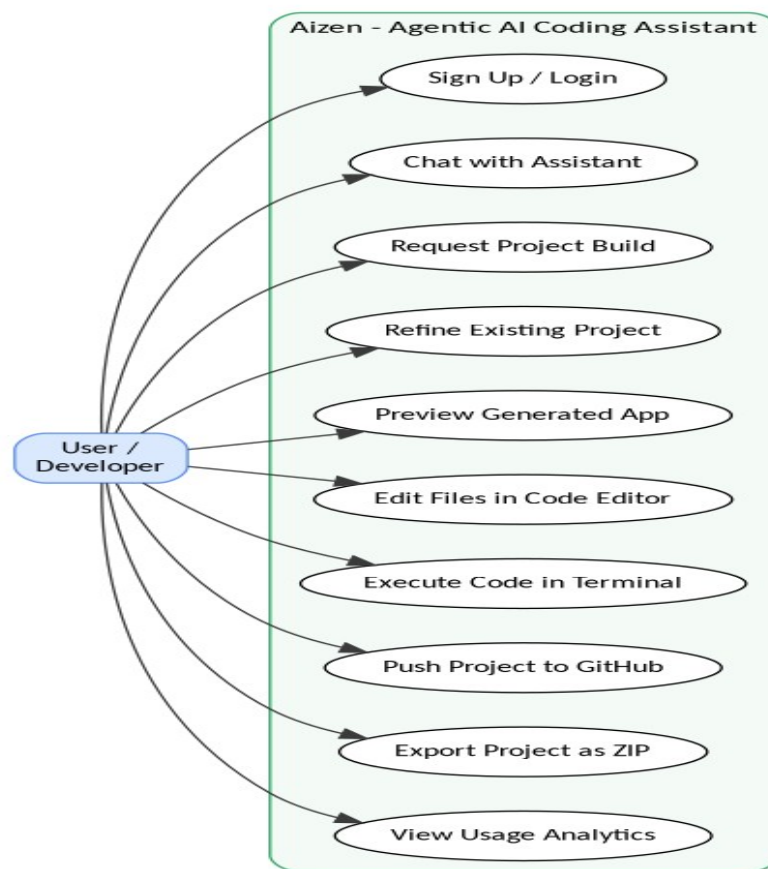


Figure 3.4: Use case diagram - Aizen

CHAPTER 4

SYSTEM REQUIREMENTS

The requirements below split into software and hardware, and the hardware half splits further into what's needed on the client, the browser running the React app, versus the server, the machine running the Flask backend and the agent loop. The two sides carry very different loads, which is really why they're split out. Client-side, it's mostly just rendering the UI and running the Monaco editor. Server-side is where the model calls, file generation and validation actually happen.

4.1 Software Requirements

Table 4.1: Software requirements

Operating System	Windows / Linux / macOS
Frontend	React 19, Vite 6, Node.js 18+, Tailwind CSS v4
Backend	Python 3.10+, Flask 3.x, Flask-CORS
AI Integration	Groq chat-completions API
Database	SQLite
Authentication	PyJWT, Werkzeug (bcrypt hashing)
IDE / Tools	Visual Studio Code, Git / GitHub
Browser (Client)	Chrome / Edge / Firefox (latest versions)

4.2 Hardware Requirements

Hardware demands stay light on both sides here, mainly because the actual model inference runs on Groq's hosted infrastructure rather than on the user's machine or the local Flask server. What the local machine needs is mostly headroom, enough to run a browser tab with an embedded code editor comfortably. On the server side it's more about disk space, holding several per-user workspaces at once, plus enough breathing room to make outbound API calls without everything slowing down.

For Client (User side):

Table 4.2: Client-side hardware requirements

Processor	Intel i3 or above (or equivalent AMD)
RAM	Minimum 4 GB (8 GB recommended)
Storage	200 MB free space for cache / browser storage

Internet	Stable connection (2 Mbps or above)
----------	-------------------------------------

For Backend Server (local Flask process):

Table 4.3: Server-side hardware requirements

Processor	Intel i5 or higher (or equivalent)
RAM	8 GB or above
Storage	1 GB minimum for workspaces and dependencies
Network	Outbound HTTPS access to the Groq API

CHAPTER 5

EXPECTED OUTCOMES...(WITH GUI)

This chapter goes screen by screen through the interface, roughly following the order a new user would actually hit them in: landing page, sign-up, the main chat/build workspace, the code editor and preview, then the two extra tools sitting alongside it, analytics and the SQL Assistant. Every screen below carries a screenshot pulled from the running application.

5.1 Landing Page

The landing page introduces Aizen, says what it does in a single line, and gives a direct path into the chat workspace. Keeping it simple was a deliberate choice, it doesn't try to walk anyone through the agent loop or the module architecture from Chapter 3. It just gets a first-time visitor into the chat interface as fast as it can, on the theory that this product makes more sense once you're using it than it ever will on a page describing it.

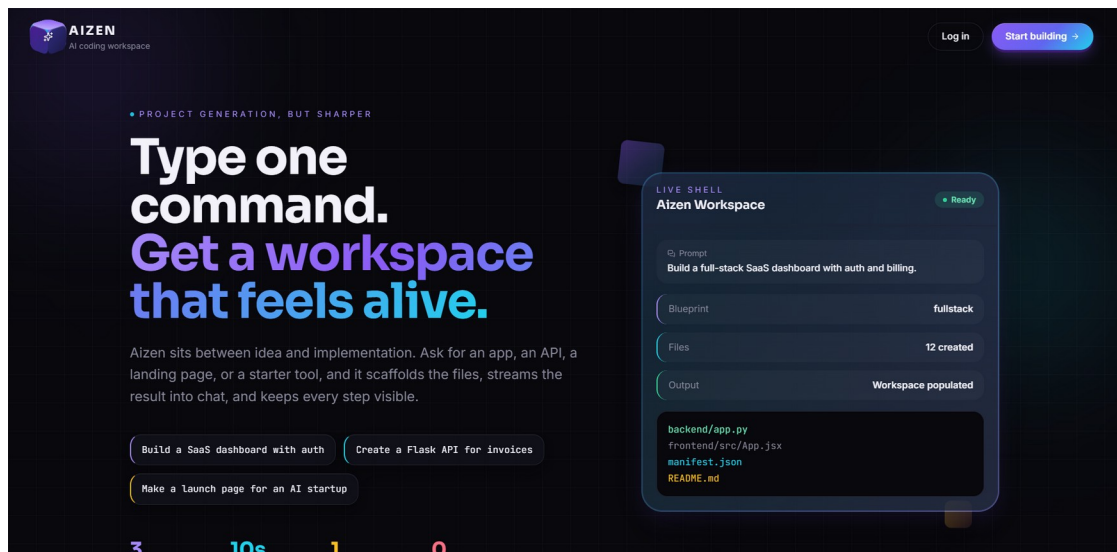


Figure 5.1: Aizen landing page

5.2 Sign-up / Login Interface

A new user can create an account or log into an existing one. From there, the session gets carried by a JWT stored on the client, which is what lets the sidebar pull up a user's own chat history the next time they come back. Passwords never touch storage as plain text, they go through

Werkzeug's bcrypt-based helper before they ever reach SQLite, so even someone with direct access to the database file wouldn't walk away with a usable password.

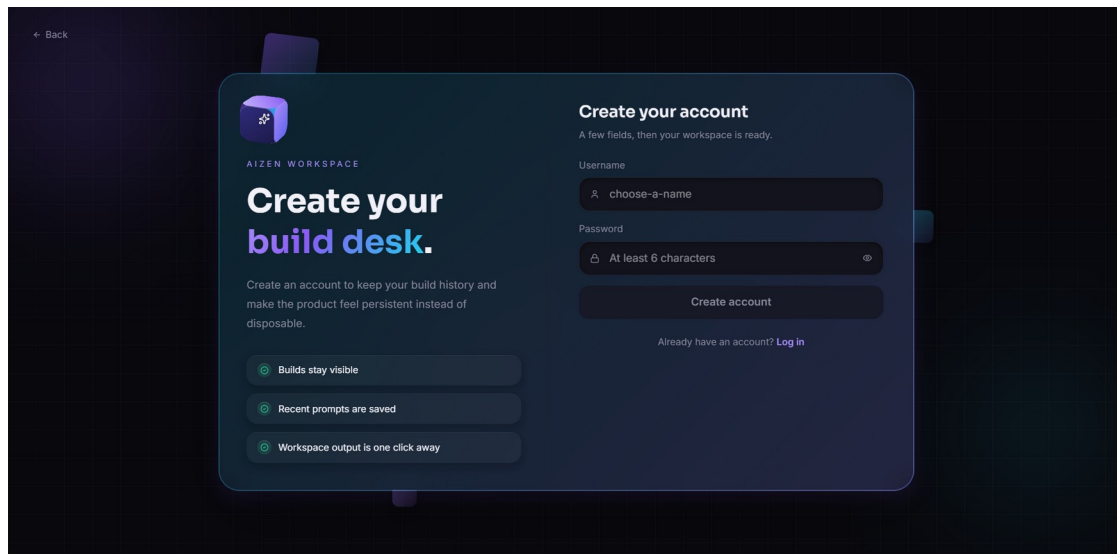


Figure 5.2: Sign-up / login interface

5.3 Chat-Based Build Workspace

The main workspace puts the conversation on the left, and once a project exists, the generated files, live preview, code editor, search and terminal panels on the right. The user keeps talking to the agent in the same thread to ask for changes, and the agent applies them to the same workspace rather than starting over. This is really where the six-stage methodology from Chapter 3 becomes visible, the streamed response, the inline project manifest, the validation note, all of it shows up here as the agent works through classification, generation, the tool-use loop and delivery.

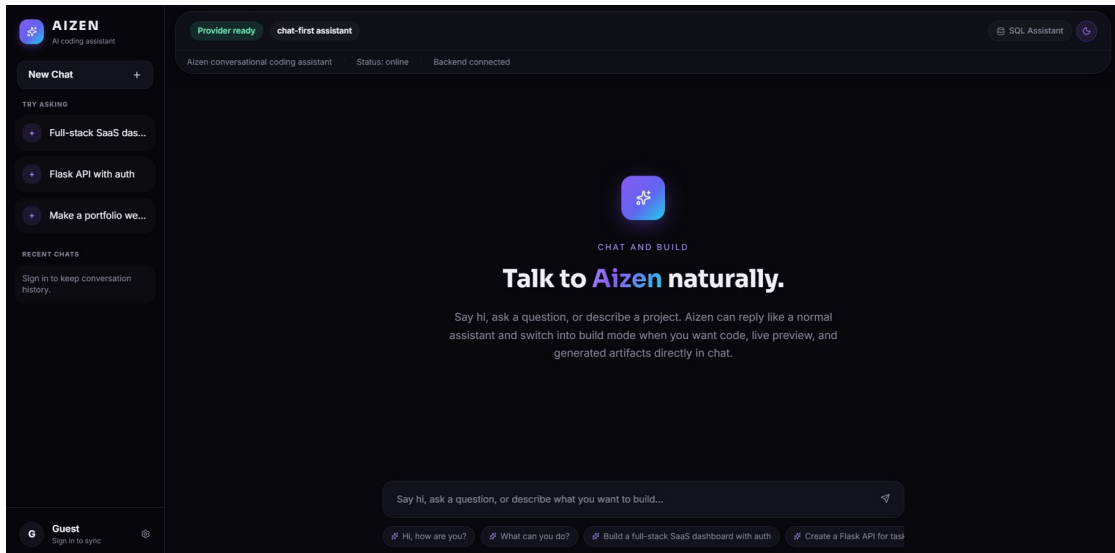


Figure 5.3: Chat-based build workspace

5.4 Live Code Editor and Preview

Generated files open straight in an in-browser Monaco editor, get edited, and save back into the workspace; HTML-renderable projects also get an instant preview in the panel next to it. Monaco is the same editor VS Code is built on, so syntax highlighting, bracket matching and basic language support came for free instead of something we had to build ourselves.

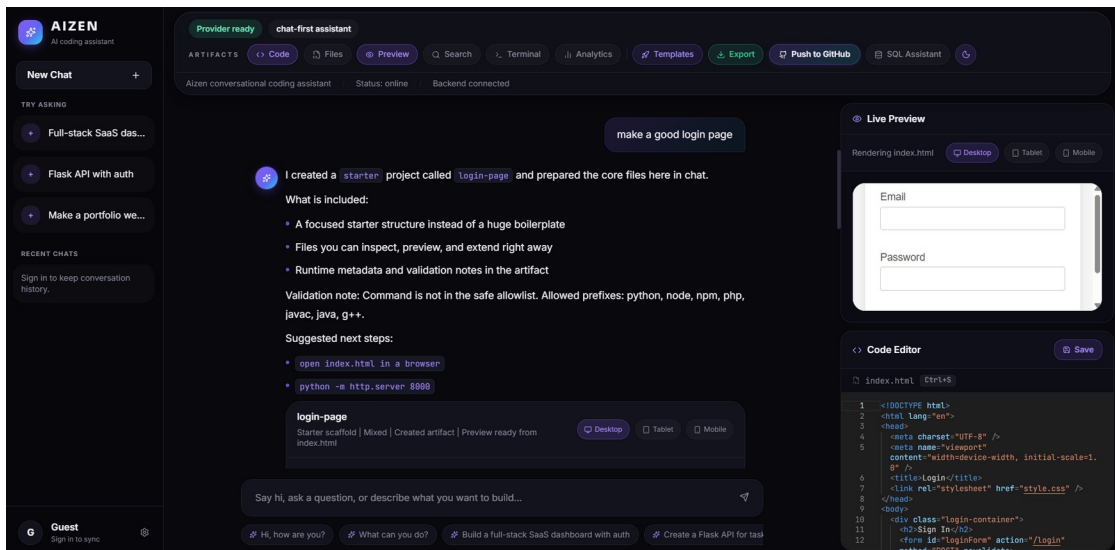


Figure 5.4: Code editor and live preview

5.5 Analytics Dashboard

A simple analytics view tracks how many projects and files a given user has generated, and what mix of languages shows up across them, mostly just so there's some visibility into how the tool actually gets used over time. It's read-only. It reports what happened and leaves it there, no attempt to recommend what to build next.

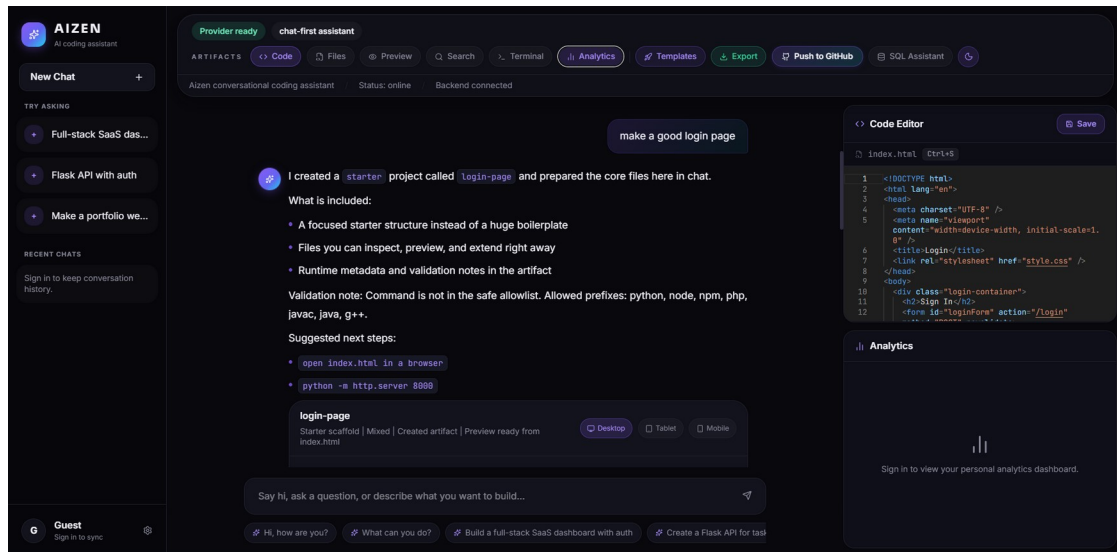


Figure 5.5: Analytics dashboard

5.6 SQL Assistant Panel

A separate panel lets the user optionally describe a table schema, pick a target SQL dialect, and ask a question in plain English, then returns a syntax-highlighted SQL query with a short explanation of what it does. It's built to the same design as the other tool panels, and opens off that same toolbar. Nobody has to fill in the schema field, the assistant can still work out a reasonable query from the question by itself. But handing it a schema does noticeably sharpen the result, since the model then has real column and table names in front of it instead of having to guess at them.

EXPECTED OUTCOMES...(WITH GUI)

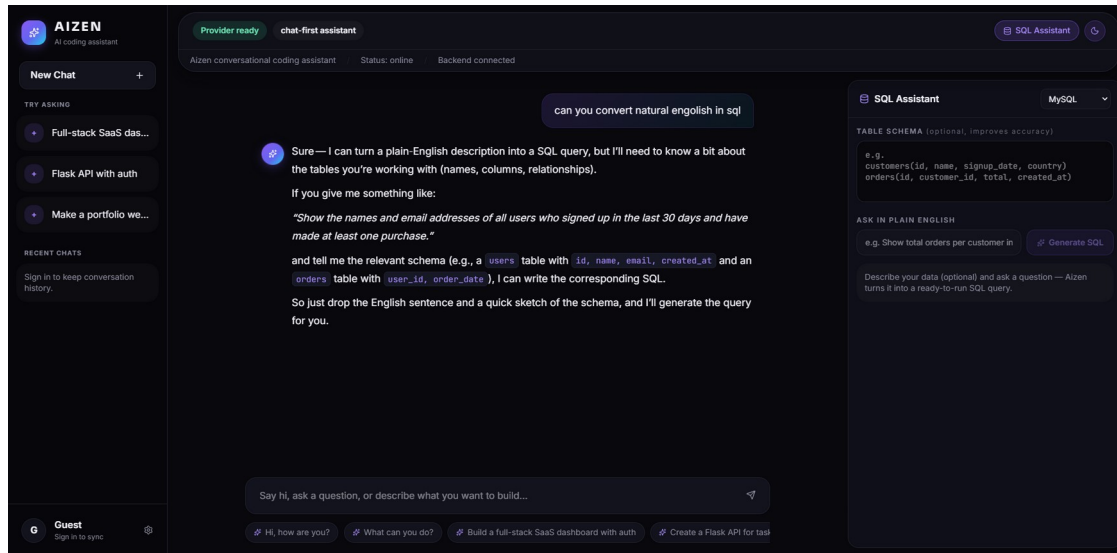


Figure 5.6: SQL Assistant panel

CHAPTER 6

CONCLUSION & FUTURE SCOPE

6.1 Conclusion

What this project really did was take the agentic-AI idea from our minor project, mostly diagrams at that point, and turn it into something that actually runs, inside a domain narrow enough that we could genuinely tell whether it works or not: generating and maintaining software projects from a single instruction. Everything described across this report is what we actually finished, the intent classifier, the project builder spanning ten stacks, the bounded tool-use loop, the validation step, the deterministic fallback, plus the workspace wrapped around all of it, editor, live preview, terminal, GitHub push, SQL Assistant.

Building this end to end surfaced problems that never showed up while the idea only lived on paper. The clearest one hit during a stretch of development where the assistant kept handing back the exact same generic starter project no matter what we asked it for. Took a while to pin down. Turned out to be two separate things happening at once: the intent classifier was missing some legitimate ways of phrasing a build request, and, unrelated to that, a language-model call was failing silently and quietly dropping to the template generator without logging why. Neither problem was visible from the outside. We only found the actual cause after deliberately reproducing the failure ourselves and digging through the backend logs, then rewrote the exception handling so it reports what actually went wrong instead of swallowing the error and returning a fallback without a word. That fix taught us something the design phase genuinely never could have: to a user, an agent that fails silently is indistinguishable from one that never worked in the first place. Handling failure out loud ended up mattering just as much as the planning and generation logic itself did.

On the limitations side, Aizen still runs on a single local Flask process and SQLite, so it isn't built for multiple users working on the same project at once, and the execution sandbox for running code isn't isolated in a container yet. Those are the two biggest gaps between what exists now and something closer to a deployable product, and they're where the future work below picks up.

On the whole, we think the project meets what we set out to do in Chapter 1: instruction classification, multi-stack project generation with a working fallback, an iterative agent loop, and

an integrated browser environment for editing, running and shipping the result. It also answers the question our minor project left open, that an agentic, tool-using approach like this can actually get built and hold up under real use, as long as its failure paths get taken as seriously as the happy path does.

6.2 Future Work

The items below didn't get built, not because we forgot about them, but because keeping the project small enough for two people to finish in the time available meant something had to stay out of scope. Here's what we'd add next:

- **Smarter fallback and self-healing** – retry with a different model or a narrower prompt before giving up and dropping to the static template. Right now, the fallback described in Section 3.3, Stage 3, always hands the user something usable, but it makes no attempt to recover the original request. A narrower retry first would let more of these requests actually succeed on the real AI-generated path instead of falling back.
- **Multi-agent collaboration** – split generation and validation across two agents, one writing the code, the other actively trying to break it, before anything reaches the user. Stage 5's single validation command would essentially grow into a proper multi-step check.
- **Stronger execution sandboxing** – move generated and user-submitted code into an isolated container instead of running it on the host process directly. The terminal feature carries real risk as it stands, and that risk only grows once more than one trusted local user shares the system.
- **Voice-based interaction** – let the user speak instructions instead of typing them.
- **Team and workspace sharing** – open up the same generated project to more than one user at a time. That would also force a move off SQLite, already noted as a limitation back in Chapter 4, toward something built to handle concurrent access properly.
- **Mobile-friendly client** – a responsive or dedicated mobile version so the assistant isn't tied to a desktop browser.

Put together, these six items form a fairly natural next phase for the project: solidify the single-user core that already exists, then grow outward toward collaboration and reach. Reliability

before scale, in other words, and that ordering comes directly out of the silent-failure problem described above. There's simply more value in making what already exists trustworthy than in piling new features onto something that hasn't been fully verified yet.

CHAPTER 7

REFERENCES

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. & Cao, Y., 2022. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629.
- [2] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. & Zhou, D., 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: Advances in Neural Information Processing Systems (NeurIPS 2022).
- [3] LangChain, 2024. LangChain Documentation. [online] Available at: <https://python.langchain.com> [Accessed 2026].
- [4] Significant Gravitas, 2023. AutoGPT: An Autonomous GPT-4 Experiment. [online] Available at: <https://github.com/Significant-Gravitas/AutoGPT> [Accessed 2026].
- [5] OpenAI, 2024. Assistants API Documentation. [online] Available at: <https://platform.openai.com/docs/assistants> [Accessed 2026].
- [6] Microsoft, 2024. Semantic Kernel Documentation. [online] Available at: <https://learn.microsoft.com/semantic-kernel> [Accessed 2026].
- [7] Groq Inc., 2026. Groq API Documentation. [online] Available at: <https://console.groq.com/docs> [Accessed 2026].
- [8] Pallets Projects, 2024. Flask Documentation. [online] Available at: <https://flask.palletsprojects.com> [Accessed 2026].
- [9] Meta Platforms Inc., 2025. React Documentation. [online] Available at: <https://react.dev> [Accessed 2026].
- [10] Vite, 2025. Vite Documentation. [online] Available at: <https://vitejs.dev> [Accessed 2026].
- [11] Microsoft, 2024. Monaco Editor Documentation. [online] Available at: <https://microsoft.github.io/monaco-editor> [Accessed 2026].
- [12] SQLite Consortium, 2024. SQLite Documentation. [online] Available at: <https://www.sqlite.org/docs.html> [Accessed 2026].

- [13] Jones, M., Bradley, J. & Sakimura, N., 2015. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force.
- [14] Pallets Projects, 2024. Werkzeug Documentation. [online] Available at: <https://werkzeug.palletsprojects.com> [Accessed 2026].
- [15] Tailwind Labs, 2025. Tailwind CSS Documentation. [online] Available at: <https://tailwindcss.com/docs> [Accessed 2026].